DuocUC | ONLINE



Guía de aprendizaje

Escalado y procesamiento paralelo en Spring Batch

Exp 1 – Semana 3

Desarrollo Backend III (PBY2203)

Facilitador disciplinar: Ana Karina Villagrán Ibarra

Asesor par: Ignacio Villaroel

Índice

Introducción a la semana....3
Resultado de aprendizaje....4
Conceptos relevantes....4
Preguntas activadoras....5
Actividad....5
Escalado y procesamiento paralelo en Spring Batch....6
Técnicas de escalado....6
Políticas de reintento....9
Caso práctico: FreshMart....10
Desarrollo del caso....10
Diferencias en la configuración....11
Resultado final....18
Cierre de la semana....19
Referencias....20
Lecturas de la semana....20
Apuntes....21

Introducción a la semana



Hasta ahora, has aprendido los fundamentos de Spring Batch y cómo estructurar trabajos por lotes para manejar grandes volúmenes de datos, aplicando técnicas como el procesamiento en chunks y el uso de hilos múltiples para optimizar el rendimiento del sistema. Con este nuevo desafío de esta semana llamada “Escalado y procesamiento paralelo en Spring Batch”, te adentrarás en la implementación de particiones, permitiéndote dividir el trabajo en partes más pequeñas que se procesan en paralelo, mejorando los tiempos de ejecución y la eficiencia en el manejo de datos a gran escala.

Es decir, en esta guía de aprendizaje encontrarás:

- Cómo configurar y optimizar particiones en Spring Batch para procesar grandes volúmenes de datos de manera eficiente.
- Qué estrategias de particionamiento y ejecución paralela se pueden implementar para reducir los tiempos de procesamiento.
- Cómo configurar políticas de reintento que garanticen la resiliencia del sistema en caso de errores o fallos temporales.

Además, explorarás la configuración de políticas de reintento, esenciales para que el sistema sea resiliente ante errores, permitiendo reintentar fallos de forma controlada y asegurando la continuidad del procesamiento batch sin interrumpir el flujo de datos. Con estas herramientas, estarás mejor preparado para enfrentar los desafíos en la optimización de sistemas batch, manteniendo la estabilidad y confiabilidad del sistema en situaciones de alta carga y fallos eventuales.

Resultado de aprendizaje

El estudiante será capaz de:

RA1. Desarrolla componentes batch para la migración de sistemas antiguos basados en cobol/shell.

Indicador de logro:

IL2. Ejecuta jobs y steps en Spring Batch, para transformar datos de entrada y generar una salida.

IL3. Implementa escalado, procesamiento paralelo y diversas políticas de jobs en procesos batch para optimizar tiempos de ejecución, estabilidad y continuidad del sistema.

Conceptos relevantes

Partition	Manager	Worker
SkipListener	Procesamiento paralelo	

Preguntas activadoras

- ¿Qué conocimientos tienes sobre el concepto de particiones?
- ¿Cómo crees que la técnica de particiones puede ayudar a mejorar la eficiencia en el procesamiento de grandes volúmenes de datos en sistemas batch?
- ¿Qué ventajas ofrece configurar políticas de reintento frente a fallos en comparación con el reinicio completo del trabajo batch?

Actividad

En esta tercera semana realizarás una actividad sumativa individual llamada "Optimizando procesos batch para mejorar la resiliencia de procesos”. En esta oportunidad, deberás desarrollar un flujo de procesamiento batch que implemente técnicas de escalado y procesamiento paralelo para mejorar los tiempos de ejecución, asegurando que los jobs manejen adecuadamente errores y excepciones, y aplicando políticas de finalización, repetición y reejecución de jobs, garantizando la resiliencia del sistema batch ante posibles fallos durante su ejecución.

Escalado y procesamiento paralelo en Spring Batch

El uso de particiones en Spring Batch es una técnica esencial para optimizar la ejecución de procesos batch, distribuyendo la carga de trabajo entre múltiples hilos o nodos. Esto mejora significativamente los tiempos de procesamiento y la eficiencia del sistema al permitir que distintas partes de un mismo trabajo se ejecuten simultáneamente. Al implementar particiones, puedes dividir grandes volúmenes de datos en secciones más pequeñas que se procesan en paralelo, asegurando un uso óptimo de los recursos y permitiendo al sistema gestionar de forma efectiva cargas intensivas de datos.

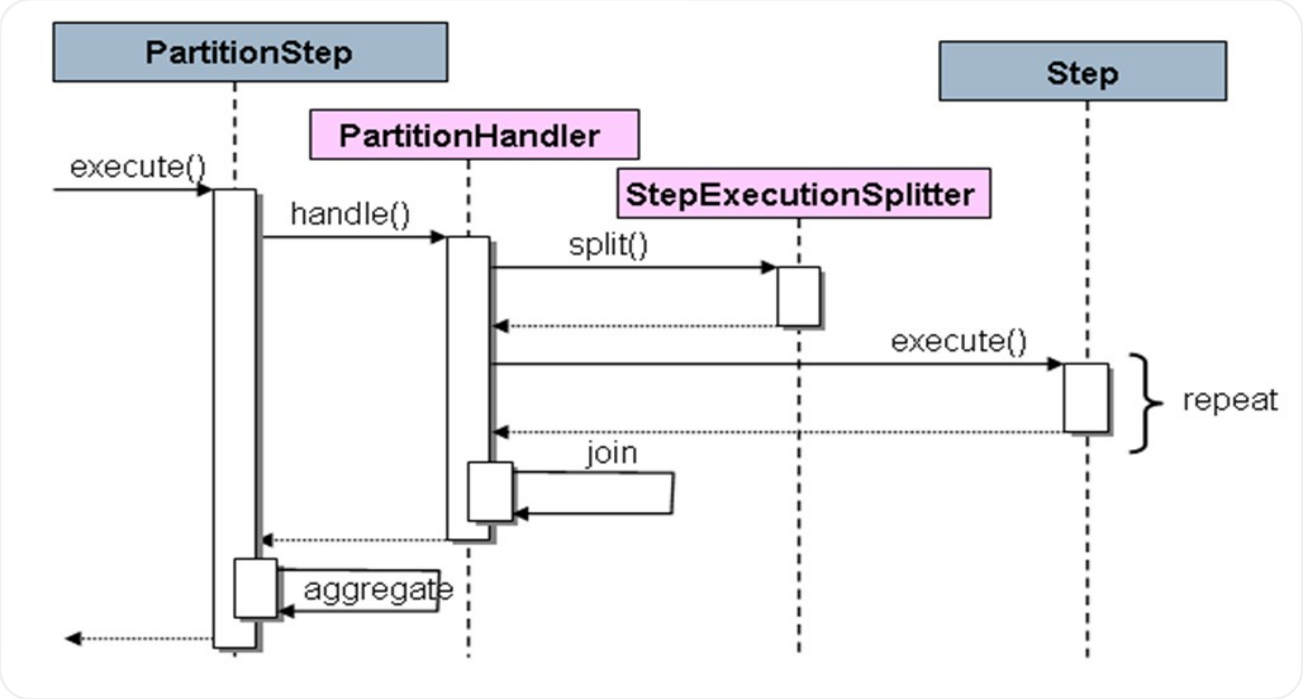
Técnicas de escalado

Particiones

El particionado en Spring Batch permite dividir un conjunto de datos en secciones independientes llamadas particiones, que pueden procesarse en paralelo para mejorar la eficiencia y el rendimiento. Cada partición actúa como una unidad lógica de procesamiento y se asigna a diferentes hilos o nodos, distribuyendo así la carga de trabajo de manera equilibrada. Esta técnica permite manejar grandes volúmenes de datos de forma más efectiva y aprovechar mejor los recursos del sistema. Además, el particionado facilita el control de errores, ya que cualquier fallo dentro de una partición específica se maneja de manera aislada sin afectar el procesamiento completo del trabajo, lo que incrementa la estabilidad y resiliencia del sistema batch.

Figura1

Procesamiento por particiones



Nota. Flujo de ejecución en un PartitionStep en Spring Batch, donde un PartitionHandler divide (split) y ejecuta en paralelo múltiples Steps, luego agrega (aggregate) los resultados. SpringSource (s.f.). *Scaling and Parallel Processing.* https://docs.spring.io/spring-batch/reference/scalability.html

En Spring Batch, el proceso de particiones permite dividir un paso (Step) en múltiples sub-tareas que se ejecutan en paralelo, optimizando el procesamiento de grandes volúmenes de datos. A continuación, te explico el flujo general del proceso:

- **PartitionStep:** es el paso principal que inicia la ejecución y gestiona el proceso de particionado.
- **PartitionHandler:** este componente es responsable de dividir el trabajo en particiones. Recibe una lista de sub-tareas y las asigna a diferentes hilos o nodos, distribuyendo la carga de trabajo.

- **StepExecutionSplitter:** el PartitionHandler utiliza el StepExecutionSplitter para dividir el Step en particiones individuales. Cada partición es un fragmento lógico del trabajo total, diseñado para ser procesado de manera independiente.
- **Ejecución en paralelo de steps:** cada partición se asigna a un Step que se ejecuta en paralelo. Esto permite procesar varias partes del trabajo al mismo tiempo, lo cual mejora la eficiencia y reduce el tiempo de ejecución.
- **Join y Aggregate:** una vez que todos los Steps de las particiones han sido ejecutados, sus resultados son combinados (aggregate) y devueltos al PartitionStep. Este proceso garantiza que los datos finales se consoliden, obteniendo un resultado unificado.

Este enfoque asegura que el sistema batch aproveche al máximo los recursos disponibles, permitiendo una mejor distribución de la carga de trabajo y la capacidad de manejar grandes volúmenes de datos de manera más eficiente. Además, si ocurre un error en una partición específica, solo esa partición necesita ser reintenta, manteniendo el resto del proceso sin interrupciones.

Políticas de reintento

Las políticas de reintento en Spring Batch son configuraciones que permiten definir el cómo manejar errores temporales o intermitentes durante la ejecución de un proceso batch. Cuando ocurre un fallo en un Step, una política de reintento especifica refiere a cuántas veces y en qué condiciones se intentará nuevamente el procesamiento antes de marcar el Step como fallido.

Estas políticas son útiles para mejorar la resiliencia del sistema, permitiendo que el proceso se recupere automáticamente ante fallos esporádicos sin interrumpir todo el trabajo. Algunas de las configuraciones más comunes de reintento incluyen:

- **RetryPolicy:** define cuántas veces se debe intentar nuevamente un Step en caso de error y establece las excepciones específicas que deben activar el reintento.
- **BackOffPolicy:** controla el tiempo de espera entre cada intento, permitiendo agregar un intervalo progresivo (como un incremento exponencial) para evitar sobrecargar el sistema si el problema persiste.
- **SkipPolicy:** configura el sistema para omitir ciertos registros que fallan, permitiendo que el proceso continúe sin reintentar los mismos registros problemáticos.

Con estas políticas, puedes ajustar la tolerancia a fallos en el sistema batch, logrando una mayor estabilidad y reduciendo las interrupciones en el flujo de datos. Además, al configurar políticas de reintento específicas, el sistema es capaz de continuar procesando, reintentar cuando sea necesario, o ignorar los errores sin afectar el trabajo completo.

Caso práctico: FreshMart

¿Recuerdas el caso de FreshMart de la semana anterior? Recordemos su contexto:

FreshMart, una cadena de supermercados en rápida expansión enfrenta problemas de lentitud y errores en la consolidación de datos debido a un sistema de procesamiento de datos obsoleto, basado en scripts batch manuales. La alta generación de datos diarios sobre ventas, inventario y proveedores exige un sistema más ágil y confiable. Para optimizar su eficiencia operativa y asegurar decisiones basadas en datos actualizados, FreshMart ha decidido modernizar su sistema utilizando Spring Batch, con el objetivo de gestionar grandes volúmenes de datos de manera rápida y eficiente.

El requerimiento del proyecto era:

Realizar una consolidación diaria de ventas, procesando cada noche los datos de ventas de todas las sucursales para generar un informe con tendencias de compra, productos más vendidos y aquellos de baja rotación. Dado que los datos provienen de distintos sistemas de punto de venta, es esencial procesarlos en paralelo para agilizar la consolidación.

Desarrollo del caso

Ahora, te invitamos a que vuelvas a revisar el caso de FreshMart, pero en esta ocasión, desarrollando la solución considerando particiones.

Links de interés

Accede al siguiente proyecto avanzado de Spring Batch para poder comprender, desde la implementación, el cómo se podría resolver el problema de FreshMart:
https://ava.duoc.cl/bbcswebdav/xid-5046659_1

Diferencias en la configuración

Lo primero a destacar con respecto a una configuración sin particiones es que, dado que existen dos steps definidos en el BatchConfig, necesitas indicar explícitamente cuál es el principal.

Figura 2

Diferencias de la configuración



Nota. La imagen muestra lo diferente respecto a la configuración del proyecto batch_advance_configs a través de Power Shell. Microsoft (2006). *Power Shell (7.4.5).* [Software]. https://learn.microsoft.com/en-us/powershell/

Esta configuración en Spring Batch permite dividir el procesamiento de un gran conjunto de datos en múltiples particiones, mejorando el rendimiento y la escalabilidad. Como puedes ver en la Figura 3, el partitionStep define un paso que distribuye el trabajo a través de varias instancias de minionStep, donde cada instancia procesa una partición del conjunto de datos.

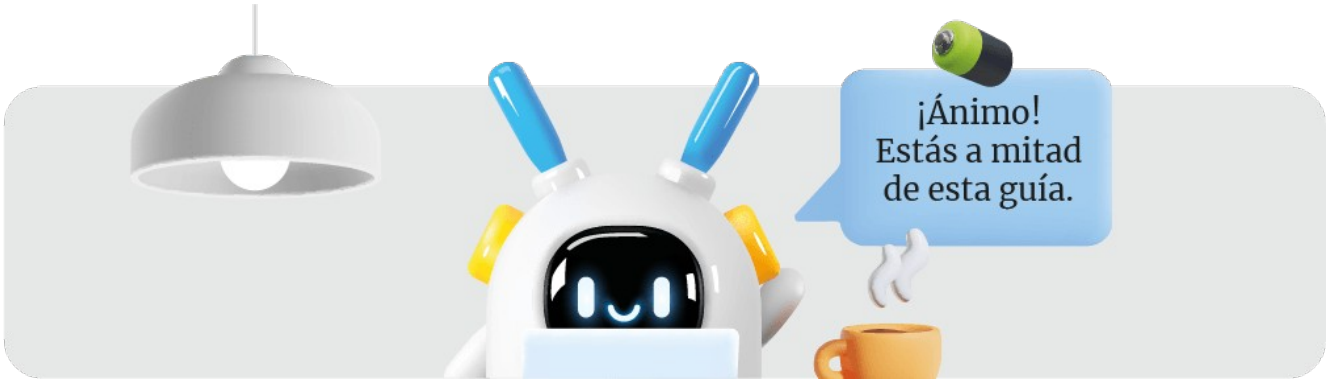
Figura 3

Step con PartitionHandler

```
@Bean
public Step partitionStep(JobRepository jobRepository,
                          TaskExecutorPartitionHandler partitionHandler,
                          Partitioner partitioner) {
    return new StepBuilder(name:"partitionStep", jobRepository)
            .partitioner(stepName:"minionStep", partitioner)
            .partitionHandler(partitionHandler)
            .build();
}
```

Nota. Nueva configuración requerida para utilizar un manejador de particiones a través de Power Shell. Microsoft (2006). *Power Shell. (7.4.5).* [Software]. https://learn.microsoft.com/en-us/powershell/

El Partitioner especifica cómo se dividen los datos, mientras que el TaskExecutorPartitionHandler permite ejecutar estas particiones en paralelo usando múltiples hilos o procesos. Esta configuración es crucial para manejar grandes volúmenes de datos de manera eficiente al aprovechar el procesamiento concurrente.



La siguiente configuración en Spring Batch define un TaskExecutorPartitionHandler, que permite ejecutar un paso en varias particiones de forma paralela. En este caso, se asocia el partitionHandler con el minionStep (el paso que se ejecutará para cada partición) y un **taskExecutor** (el ejecutor de tareas que administrará los hilos de ejecución). El método **setGridSize**(3) establece que se utilizarán 3 particiones para procesar los datos en paralelo, lo que permite distribuir la carga de trabajo y mejorar el rendimiento al aprovechar la concurrencia.

Figura 4

Configuración de PartitionHandler



```
@Bean
public TaskExecutorPartitionHandler partitionHandler(Step minionStep, TaskExecutor taskExecutor) {
    TaskExecutorPartitionHandler handler = new TaskExecutorPartitionHandler();
    handler.setStep(minionStep);
    handler.setTaskExecutor(taskExecutor);
    handler.setGridSize(gridSize:3); // Número de particiones
    return handler;
}
```

Nota. Imagen que muestra la configuración especifica de un manejador de particiones a través de Power Shell. Microsoft (2006). *Power Shell.* (7.4.5). [Software]. https://learn.microsoft.com/en-us/powershell/

La siguiente configuración es crucial, porque permite dividir el procesamiento de datos en particiones manejables, lo cual optimiza el uso de recursos y mejora el rendimiento en Spring Batch:

Figura 5

Función de partición



```
@Bean
public Partitioner partitioner() {
    return gridSize -> {
        Map<String, ExecutionContext> partitions = new HashMap<>();
        int totalData = 10;  // Cambia esto al número real de datos
        int partitionSize = (int) Math.ceil((double) totalData / gridSize);

        int start = 0;
        for (int i = 0; i < gridSize; i++) {
            ExecutionContext context = new ExecutionContext();
            int end = Math.min(start + partitionSize - 1, totalData - 1);

            context.putInt(key:"start", start);
            context.putInt(key:"end", end);
            context.putString(key:"partitionName", "partition" + i);
            partitions.put("partition" + i, context);

            start += partitionSize;
            if (start >= totalData) {
                break;
            }
        }
        return partitions;
    };
}
```

Nota. Función importante en la configuración de particiones a través de Power Shell. Microsoft (2006). *Power Shel.l* (7.4.5). [Software]. https://learn.microsoft.com/en-us/powershell/

Aquí, la clase Partitioner crea un conjunto de ExecutionContext que define rangos de datos (mediante start y end) para cada partición, asegurando que cada una procese solo una parte específica de los datos totales. Esto resulta especialmente útil en sistemas con grandes volúmenes de datos, ya que permite ejecutar varias particiones en paralelo, distribuyendo la carga de trabajo y aprovechando mejor los recursos del sistema. Además, al manejar un límite específico (totalData), la configuración evita procesar datos fuera del rango necesario, lo cual garantiza eficiencia y precisión en el procesamiento de lotes.

Debes considerar editar tu ItemReader para considerar estas configuraciones de ExecutionContext (contexto de ejecución). Para ello, revisa la nueva implementación de la clase **VentasItemReader:**

```
[main] INFO org.springframework.batch.core.configuration.annotation.BatchRegistrar - Finished Spring Batch infrastructure beans configuration in 14 ms.

[main] WARN org.springframework.context.support.PostProcessorRegistrationDelegate$BeanPostProcessorChecker - Bean 'jobRegistry' of type [org.springframework.batch.core.configuration.support.MapJobRegistry] is not eligible for getting processed by all BeanPostProcessors (for example: not eligible for auto-proxying). Is this bean getting eagerly injected into a currently created BeanPostProcessor [jobRegistryBeanPostProcessor]? Check the corresponding BeanPostProcessor declaration and its dependencies.

[main] INFO org.springframework.jdbc.datasource.embedded.EmbeddedDatabaseFactory - Starting embedded database: url='jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=false', username='sa'

[main] INFO org.springframework.batch.core.repository.support.JobRepositoryFactoryBean - No database type set, using meta data indicating: H2

[main] INFO org.springframework.batch.core.configuration.annotation.BatchObservabilityBeanPostProcessor - No Micrometer observation registry found, defaulting to ObservationRegistry.NOOP

[main] INFO org.springframework.batch.core.configuration.annotation.BatchObservabilityBeanPostProcessor - No Micrometer observation registry found, defaulting to ObservationRegistry.NOOP

[main] INFO org.springframework.batch.core.configuration.annotation.BatchObservabilityBeanPostProcessor - No Micrometer observation registry found, defaulting to ObservationRegistry.NOOP

[taskExecutor-1] INFO org.springframework.batch.core.launch.support.SimpleJobLauncher - Job: [SimpleJob: [name=partitionedJob]] launched with the following parameters: [{'time':'{value=1730656768473, type=class java.lang.Long, identifying=true}'}]

[taskExecutor-1] INFO com.duoc.advanced.JobCompletionListener - Inicio del trabajo batch con ID: 1

[taskExecutor-1] INFO org.springframework.batch.core.job.SimpleStepHandler - Executing step: [partitionStep]

[taskExecutor-3] WARN com.duoc.items.VentasItemProcessor - Precio inválido para producto Producto F: 0.0

[taskExecutor-3] INFO com.duoc.advanced.FileVerificationSkipper - Error: FlatFileParseException
```

```
[taskExecutor-3] WARN com.duoc.advanced.FileVerificationSkipper - CustomSkipPolicy - Excepción omitida: Precio debe ser mayor que 0

[taskExecutor-3] WARN com.duoc.items.VentasItemProcessor - Precio inválido para producto Producto H: 0.0

[taskExecutor-3] INFO com.duoc.advanced.FileVerificationSkipper - Error: FlatFileParseException

[taskExecutor-3] WARN com.duoc.advanced.FileVerificationSkipper - CustomSkipPolicy - Excepción omitida: Precio debe ser mayor que 0

[taskExecutor-3] INFO com.duoc.advanced.VentaSkipListener - Tipo de excepción en SkipListener: com.duoc.business.InvalidDataException

[taskExecutor-3] INFO com.duoc.advanced.VentaSkipListener - Tipo de excepción en SkipListener: com.duoc.business.InvalidDataException

[taskExecutor-2] INFO org.springframework.batch.core.step.AbstractStep - Step: [minionStep:partition1] executed in 99ms

[taskExecutor-3] INFO org.springframework.batch.core.step.AbstractStep - Step: [minionStep:partition0] executed in 103ms

[taskExecutor-2] WARN com.duoc.items.VentasItemProcessor - Precio inválido para producto Producto J: 0.0

[taskExecutor-2] INFO com.duoc.advanced.FileVerificationSkipper - Error: FlatFileParseException

[taskExecutor-2] WARN com.duoc.advanced.FileVerificationSkipper - CustomSkipPolicy - Excepción omitida: Precio debe ser mayor que 0

[taskExecutor-2] INFO com.duoc.advanced.VentaSkipListener - Tipo de excepción en SkipListener: com.duoc.business.InvalidDataException

[taskExecutor-2] INFO org.springframework.batch.core.step.AbstractStep - Step: [minionStep:partition2] executed in 47ms

[taskExecutor-1] INFO org.springframework.batch.core.step.AbstractStep - Step: [partitionStep] executed in 192ms

[taskExecutor-1] INFO com.duoc.advanced.JobCompletionListener - Job completado exitosamente con ID: 1

[taskExecutor-1] INFO com.duoc.advanced.JobCompletionListener - Resumen del Job: JobExecution: id=1, version=1, startTime=2024-11-03T14:59:28.542909, endTime=2024-11-03T14:59:28.777158800, lastUpdated=2024-11-03T14:59:28.542909, status=COMPLETED, exitStatus=exitCode=COMPLETED;exitDescription=, job=[JobInstance: id=1, version=0, Job=[partitionedJob]], jobParameters=[{'time':'{value=1730656768473, type=class java.lang.Long, identifying=true}'}]

[taskExecutor-1] INFO org.springframework.batch.core.launch.support.SimpleJobLauncher - Job: [SimpleJob: [name=partitionedJob]] completed with the following parameters: [{'time':'{value=1730656768473, type=class java.lang.Long, identifying=true}'}] and the following status: [COMPLETED] in 234ms

[main] INFO org.springframework.jdbc.datasource.embedded.EmbeddedDatabaseFactory - Shutting down embedded database: url='jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=false'
```

En el nuevo log generado, se observa el uso de particiones para dividir el trabajo en partes más pequeñas y procesarlas en paralelo. Esto se hace mediante pasos llamados partitionStep y minionStep, que distribuyen el procesamiento en varios hilos de ejecución (taskExecutor-1, taskExecutor-2, etc.), permitiendo manejar grandes volúmenes de datos de forma más eficiente.

En ambas implementaciones se utilizaron mecanismos para manejar errores llamados SkipListener y CustomSkipPolicy. Estos permiten que el proceso continue, aunque encuentre datos problemáticos, como registros con precios inválidos o campos incompletos. En lugar de detenerse, el proceso "omite" estos errores y sigue trabajando con los registros válidos.

Y como último dato importante, la ejecución lineal vista la semana anterior tomo en tiempo 329ms vs la implementación actual de 234ms. Si bien el archivo de prueba es bastante pequeño, esto es un indicio de que impacto podría tener en grandes volúmenes de datos.

En resumen, la configuración con particiones es más adecuada para trabajos grandes y complejos, ya que distribuye la carga y permite un procesamiento más rápido al dividir el trabajo. La ejecución secuencial es más simple y puede ser suficiente para trabajos pequeños, pero no aprovecha las ventajas de la ejecución en paralelo. En ambas estrategias puedes incluir el manejo de errores que permiten que el proceso continúe incluso si hay datos problemáticos, pero la configuración de particiones lo hace de manera más distribuida, lo cual es útil para mantener la estabilidad en grandes volúmenes de datos.

Resultado final

Una vez que ejecutes el proceso con éxito, podrás encontrar un archivo de salida, el cual en la Figura 6 se denomina output.csv. Este archivo representará tus datos transformados según las reglas de negocio que hayas implementado.

Figura 6

Nuevo resultado del proceso



```
@Bean
public Partitioner partitioner() {
    return gridSize -> {
        Map<String, ExecutionContext> partitions = new HashMap<>();
        int totalData = 10;  // Cambia esto al número real de datos
        int partitionSize = (int) Math.ceil((double) totalData / gridSize);

        int start = 0;
        for (int i = 0; i < gridSize; i++) {
            ExecutionContext context = new ExecutionContext();
            int end = Math.min(start + partitionSize - 1, totalData - 1);

            context.putInt(key:"start", start);
            context.putInt(key:"end", end);
            context.putString(key:"partitionName", "partition" + i);
            partitions.put("partition" + i, context);

            start += partitionSize;
            if (start >= totalData) {
                break;
            }
        }
        return partitions;
    };
}
```

Nota. Captura que muestra salida final del batch con particiones. En este caso, como el control de errores se maneja en cada partición, la salida del proceso crea 3 archivos de errores. Microsoft. (2024). *Visual Studio Code.* (1.95). [Software]. https://code.visualstudio.com/

Cierre de la semana



Al cerrar esta semana, has dado un paso significativo en la configuración y optimización de procesos batch en Spring Batch con el uso de particiones. Ahora puedes dividir un proceso en múltiples particiones, lo que te permite distribuir eficientemente el trabajo entre varios hilos, manejando grandes volúmenes de datos con mayor rapidez y aprovechando mejor los recursos del sistema. Implementaste un PartitionHandler y configuraste particionadores para distribuir la carga de trabajo, permitiendo que cada partición procese una parte específica de los datos, lo cual reduce significativamente los tiempos de ejecución y mejora el rendimiento global del sistema.

También has aprendido a gestionar la continuidad de los procesos batch en un entorno particionado, aplicando políticas de tolerancia a fallos y reintentos en cada partición de forma individual. Esta capacidad asegura que, en caso de errores en una partición, solo esa parte del proceso sea reintentada o manejada, sin afectar al resto del sistema. Esto garantiza una mayor resiliencia, ya que los procesos pueden recuperarse y continuar sin comprometer la integridad de los datos, incluso en casos de fallos parciales.

Finalmente, has fortalecido tu habilidad para optimizar el uso de recursos en el procesamiento en paralelo, gestionando el manejo de errores y ajustando el tamaño de los chunks de manera que cada partición trabaje de forma eficiente. Esta experiencia en el manejo de particiones en Spring Batch te permite construir sistemas batch más robustos, escalables y preparados para soportar grandes cargas de datos, maximizando la eficiencia y confiabilidad en entornos de procesamiento intensivo.

Referencias

- Spring Batch. (s.f.). *Spring batch architecture.* https://docs.spring.io/spring-batch/reference/spring-batch-architecture.html

Lecturas de la semana

- Refactoring Guru. (s.f.). *Chain of Responsibility.* https://refactoring.guru/es/design-patterns/chain-of-responsibility
- Refactoring Guru. (s.f.). *Command.* https://refactoring.guru/es/design-patterns/command
- Spring. (s.f.). *Common Batch Patterns. Spring.* https://docs.spring.io/spring-batch/reference/common-patterns.html#commonPatterns
- Spring. (s.f.). *Unit Testing Spring.* https://docs.spring.io/spring-batch/reference/testing.html

Apuntes



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC
La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.